

Technical Interview Planning Document

Problem Restatement and Key Constraints

Problem: Given a sequence of values, remove consecutive duplicate elements so that only the first occurrence of each repeated group remains. The order of elements must stay the same. The solution must be implemented using custom data structures (nodes and pointers), without using built-in list manipulation methods.

Key Constraints:

- Input is a sequence (i will treat it as a linked structure)
- Output must preserve the original order
- Only consecutive duplicates are removed
- Must use:
 - Nodes
 - Pointers
 - Custom data structures (linked list)
- Cannot use built-in methods like `.remove()`, slicing, etc.

Clarifying Questions to Ask:

- Is the input always a non-null?
- Can the input be empty?
- Are values always integers, or can they be any type?
- Should the output be returned as a list or linked list?
- Are we modifying in-place or creating a new structure?

Inputs and Expected Outputs

Example Input 1:

None

[2,2,3,3,3,4,5,5]

Expected Output: [2,3,4,5]

Example Input 2

```
None  
[1,1,1,1]
```

Expected Output: 1

Example Input 3 (Edge Case):

```
None  
[]
```

Expected Output: []

Example Input 3 (Edge Case):

```
None  
[1]
```

Expected Output: [1]

Example Input 3 (Edge Case):

```
None  
[1,2,3,4]
```

Expected Output: [1,2,3,4]

Approach 1: Brute Force

Algorithm Description: Traverse the input and build a new linked list. Add the first element, and for each next element, only add if it is different from the previous one.

Pseudocode:

```
None  
create new linked list result  
prev_value = None
```

```
for each value in input:
    if value != prev_value
        add value to result
    prev_value = value

return result
```

Time Complexity: $O(n)$ → We traverse the list once

Space Complexity: $O(n)$ → We create a new list

Why Start with This Approach:

- Simple and safe
 - Easy to explain in an interview
 - Helps validate logic before optimizing
-

Approach 2: Optimized (In-place Linked List)

Algorithm Description: Traverse the linked list using a pointer. If the current node has the same value as the next node, skip the next node by updating pointers.

Pseudocode:

```
None
current = head

while current and current.next:
    if current.value == current.next.value
        current.next = current.next.next #skipping duplicate
    else:
        current = current.next

return head
```

Time Complexity: $O(n)$ → Single traversal

Space Complexity: $O(1)$ → No extra data structures

Why This is Better:

- Uses constant space
 - Modifies structure in-place
 - More efficient and shows mastery of pointers
-

Key Communication Points

What to Emphasize During Explanation:

1. **Problem Clarification Phase:**
 - Emphasize that duplicates must be consecutive
 - Clarify structure: array vs linked list
 - Ask about empty input and return type
 2. **When Discussing Your Approach:**
 - "Since we only care about consecutive duplicates, we don't need hashing"
 - "This becomes a pointer traversal problem"
 3. **When Explaining Your Code:**
 - Need to highlight:
 - i. Pointer movement
 - ii. Why we skip nodes
 - iii. How we avoid losing references
 4. **Complexity Trade-offs:**
 - Brute force → easier but uses extra space
 - Optimized → same speed but better memory
-

Example Walkthrough Plan

Use this example during the interview:

None

[2,2,3,3,3,4]

Step-by-step explanation:

Use this example during the interview:

None

[2, 2, 3, 3, 3, 4]

Start:

None

2 → 2 → 3 → 3 → 3 → 4

^

current

- Compare 2 and 2, since duplicate skip

None

2 → 3 → 3 → 3 → 4

^

- Compare 2 and 3, since different move forward

None

2 → 3 → 3 → 3 → 4

^

- Compare 3 and 3, since duplicate skip

None

2 → 3 → 3 → 4

^

- Compare 3 and 3, since duplicate skip again

None

2 → 3 → 4

Final: 2 → 3 → 4

During walkthrough, emphasize:

- Pointer never moves when deleting
 - Only moves when values differ
 - Ensure no elements are skipped incorrectly
-

Python Implementation

Python

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

def remove_consecutive_duplicates(head):
    #Edge Case: empty list
    if head is None:
        return None

    current = head

    #Traverse the list
    while current is not None and current.next is not None:
        if current.value == current.next.value:
            #Skip duplicate node
            current.next = current.next.next
        else:
            #Move forward
            current = current.next

    return head
```

Key implementation points to mention:

- When removing a duplicate node, the current pointer should not be advanced. This ensures that the new current.next node is properly checked and no duplicates are missed.
- Advancing the pointer immediately after deletion could cause the algorithm to skip over consecutive duplicates, leading to incorrect results.
- Edge cases must be handled explicitly, including:
 - An empty list (no nodes to process)

- A single-element list (no possibility of duplicates)
- This solution demonstrates a fundamental linked list technique where pointer manipulation is used to modify the structure efficiently without requiring additional memory.

Leetcode Playground: <https://leetcode.com/playground/UMKSiMH7>